Innovación y cierre

AngloElectric Project

Integrantes del Equipo: Fiorella Osorio

Cristopher Figueroa

David Valenzuela

Kevin Peña

1.1 Innovación

1. **¿Qué problema resuelve nuestra solución?**

   Resuelve la ineficiencia operativa en el procesamiento manual de cotizaciones de sistemas solares fotovoltaicos en la Región de Valparaíso. Actualmente, las solicitudes se atienden manualmente por WhatsApp o correo electrónico calculando recibos de luz a mano, lo que genera tiempos de espera de **48 a 72 horas**, errores de cálculo humano, visitas técnicas innecesarias en etapas tempranas y pérdida de oportunidades de venta.

2. **¿Qué la diferencia de otras alternativas?**

   A diferencia de los catálogos web estáticos o formularios de contacto tradicionales que siguen dependiendo de la revisión humana, **AngloElectric** integra un motor de Machine Learning basado en el algoritmo **K-Nearest Neighbors (modelo_solar.joblib)** que realiza inferencia local en memoria dentro del backend FastAPI. Esto permite recomendar el kit fotovoltaico exacto de forma instantánea y enlazar esa recomendación directamente con el carrito de compras, el checkout y la reserva de la visita e instalación técnica en línea.

3. **¿Qué valor agrega al usuario?**

   - **Al Usuario Final (Propietario residencial o comercial):** Permite cotizar a medida en segundos las 24 horas del día ingresando su consumo en kWh, adquirir los equipos en línea y agendar su visita técnica en un solo flujo de autoservicio.

   - **Al Administrador (Equipo comercial de AngloElectric):** Elimina el cuello de botella en pre-venta, centraliza el control de stock y precios mediante un panel CRUD protegido por rol (ADMIN) y entrega métricas de ventas y márgenes en tiempo real en el Dashboard Analítico.

4. **¿Qué evidencia tenemos de ese valor?**

   - **Reducción del tiempo de cotización (Antes vs. Después):** El tiempo de entrega de un presupuesto técnico pasó de **48–72 horas** (proceso manual previo) a **menos de 1.2 segundos** (inferencia local KNN + consulta a PostgreSQL en Neon DB), reduciendo en un **99.9%** el tiempo de espera del cliente.

   - **Validación técnica del modelo predictivo:** Ejecución exitosa de pruebas de inferencia aislada mediante probar_modelo.py (Tarea T-2.6) verificando la recomendación precisa de kits fotovoltaicos (es_kit = true) según perfiles reales de consumo.

   - **Validación con contraparte y pruebas integrales:** Validación de flujos de interacción junto a la contraparte (**Christopher Pastén**, Tarea T-0.3) y aprobación del **100% de los criterios de aceptación y Definition of Done (7/7)** en las 6 Historias de Usuario (HU-01 a HU-06).

1.2 Documento de Cierre (Scrum)

1. Resumen de Sprints y Resultados Obtenidos:

- **Sprint 0 — Planificación, Arquitectura y Diseño UI/UX (Semanas 1 a 3 | 11 de agosto al 1 de septiembre):** Se realizó el levantamiento de requerimientos funcionales y no funcionales, la validación de flujos de negocio junto a la contraparte y la definición de la arquitectura tecnológica. Además, se estructuró el *Product Backlog* y se diseñaron los prototipos interactivos de alta fidelidad en Figma para el portal de clientes y el panel administrativo (Tareas T-0.1 a T-0.4). **Resultado:** 100% de los objetivos completados.

- **Sprint 1 — Arquitectura Backend, Persistencia y Seguridad (Semanas 4 a 8 | 2 de septiembre al 6 de octubre):** Se implementó el modelo de datos relacional en la nube sobre PostgreSQL (Neon DB) junto con la capa de validación de esquemas y el poblamiento inicial del inventario fotovoltaico. Asimismo, se desarrolló el módulo de autenticación mediante cifrado Bcrypt y tokens JWT, habilitando los servicios RESTful para la gestión de usuarios, catálogo y categorías (Tareas T-1.1 a T-1.6). **Resultado:** Historias **HU-01** (Autenticación y Perfiles) y **HU-03** (Catálogo — Capa Backend) finalizadas al 100%.

- **Sprint 2 — Integración Frontend, Comercio Electrónico y Motor de IA (Semanas 9 a 15 | 7 de octubre al 10 de noviembre):** Se entrenó y desplegó el modelo predictivo K-Nearest Neighbors (Scikit-Learn) en la capa de servicios para la recomendación automatizada de kits solares. En paralelo, se construyeron en Next.js las interfaces del cotizador inteligente, el catálogo interactivo de productos, la gestión de estado global del carrito de compras y el flujo transaccional de pago y confirmación de pedidos (Tareas T-2.1 a T-2.7). **Resultado:** Historias **HU-02** (Recomendador IA), **HU-03** (Catálogo — Capa Frontend) y **HU-04** (Carrito y Checkout) finalizadas al 100%.

- **Sprint 3 — Agendamiento, Analítica Administrativa y Despliegue Cloud (Semanas 16 a 18 | 11 de noviembre al 17 de noviembre):** Se desarrolló el sistema de reserva de visitas técnicas con validación de disponibilidad horaria y el panel de administración protegido por roles, integrando el módulo CRUD de inventario, el seguimiento de cotizaciones y el dashboard de métricas comerciales. Finalmente, se estandarizó e inmutabilizó toda la infraestructura mediante contenedores Docker para su despliegue en producción (Tareas T-3.1 a T-3.7). **Resultado:** Historias **HU-05** (Agenda Técnica) y **HU-06** (Panel Administrativo) finalizadas al 100%.

2. Síntesis de Retrospectivas y Mejoras Aplicadas en los Sprints:

- **Sprint 0 (Discrepancias en el flujo del cotizador IA):** Al inicio existían diferencias de criterio sobre el recorrido exacto del usuario al cotizar; se resolvió realizando sesiones de alineación y definiendo un *User Journey* unificado antes de prototipar en Figma.
- **Sprint 1 (Serialización de objetos ORM a JSON):** Se presentaron errores de validación al retornar objetos directos de SQLAlchemy hacia el frontend; se solucionó implementando esquemas estrictos de entrada y salida con **Pydantic (schemas.py)**.
- **Sprint 2 (Tipado numérico de entrada hacia el modelo KNN):** El motor de IA fallaba cuando los formularios del frontend enviaban cadenas de texto (strings) en lugar de valores numéricos; se corrigió aplicando funciones de parseo y validación estricta en el cliente antes de consumir el endpoint de predicción.
- **Sprint 3 (Bloqueo de comunicación entre contenedores por CORS):** Al empaquetar y pasar a producción los contenedores de Next.js y FastAPI, las políticas CORS bloquearon las peticiones; se resolvió configurando explícitamente los orígenes, IP y dominio permitidos en el middleware de seguridad de FastAPI (main.py).

3. Evidencia de cómo cambió el producto durante el desarrollo:

3.1. Evidencia Audiovisual de la Evolución del Producto: Para verificar de forma visual la transición evolutiva de la plataforma —desde el diseño inicial de interfaces hasta el flujo transaccional y predictivo completamente operativo con Inteligencia Artificial—, el equipo ha dispuesto un repositorio de grabaciones en video que documentan las pruebas de usuario y la ejecución del MVP:

- **Demostración de la primera semana de desarrollo**: Video_Primera_Semana.mp4
- **Demostración última semana de desarrollo:** **Video_Ultima_Semana_Implementacion.mp4**